

AI Resume Screening Assistant

Project Documentation

1. Project Overview

Project Name: AI Resume Screening Assistant

Category: Retrieval-Augmented Generation (RAG) + Agentic AI Workflow

One-line summary: An AI-powered system that automatically screens multiple candidate resumes against a job description, ranks them by relevance, and highlights missing skills — using a RAG pipeline and a LangGraph-based agentic workflow.

Problem Statement: Recruiters and campus placement teams manually read through dozens (sometimes hundreds) of resumes to shortlist candidates for a role. This is slow, inconsistent, and prone to human bias or fatigue. There is no simple, explainable tool that can quickly tell a recruiter *"this candidate is an 82% match, missing Docker and AWS"* instead of just returning a black-box score.

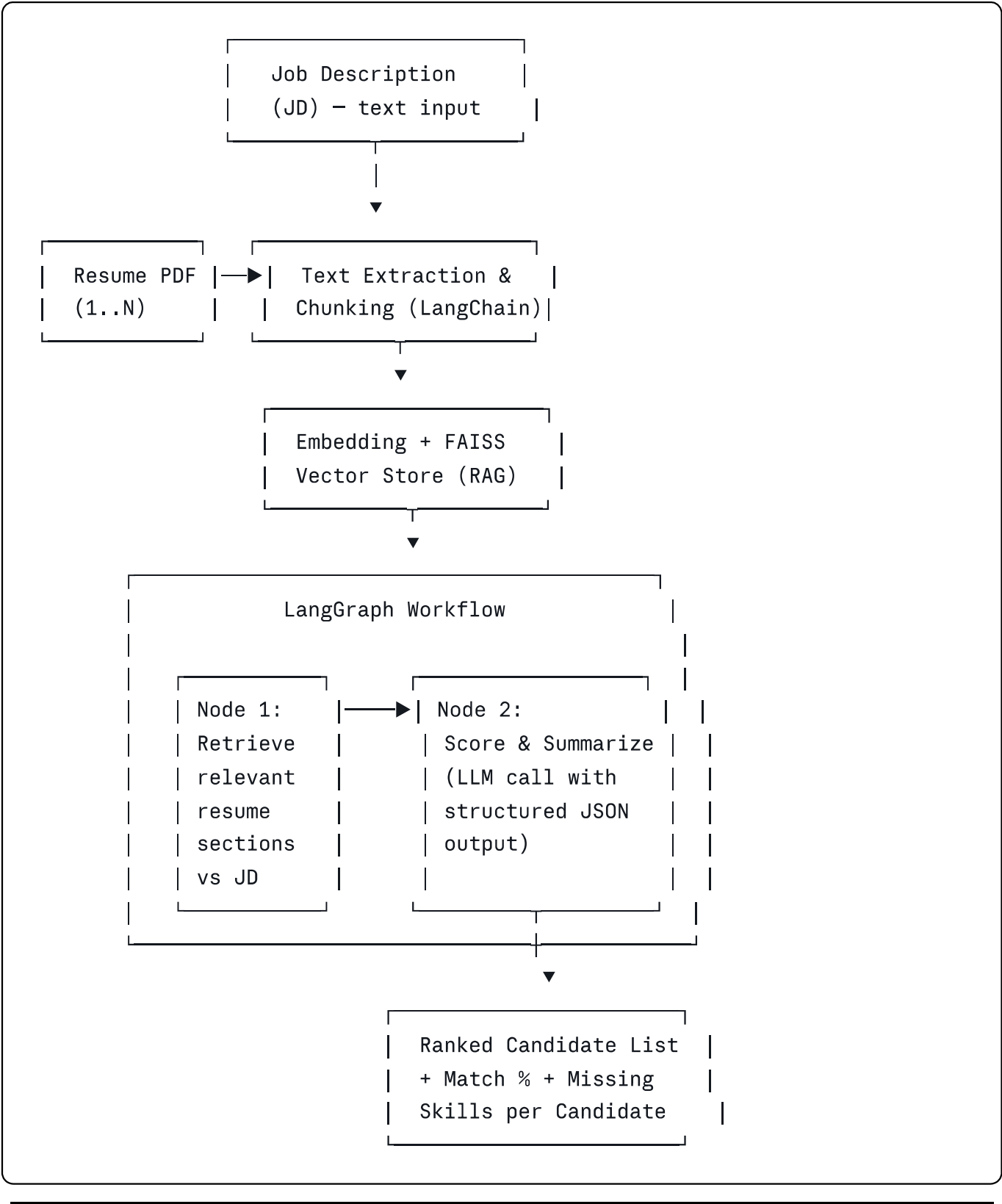
Objective: Build a lightweight AI assistant that:

1. Accepts a Job Description (JD) and a set of resumes (PDFs).
 2. Retrieves the most relevant sections of each resume against the JD using RAG.
 3. Uses an LLM, orchestrated through a LangGraph agentic workflow, to generate a structured match score and a skill-gap summary for each candidate.
 4. Outputs a ranked list of candidates with clear, human-readable reasoning.
-

2. Why This Project Fits the Role

Requirement from Job Posting	How This Project Demonstrates It
Strong Python fundamentals	Entire pipeline (PDF parsing, embedding, orchestration, scoring logic) is written in Python.
Hands-on RAG experience	Resume content is chunked, embedded, and retrieved against the JD using a vector store (FAISS) before the LLM ever sees it — the core definition of RAG.
Agentic AI workflows (LangChain/LangGraph)	The scoring process is modeled as a multi-node LangGraph graph, not a single prompt call — the system reasons in steps (retrieve → compare → score → summarize).
LLM application development	An LLM is used as the reasoning engine for scoring and generating skill-gap explanations, wrapped in a usable application (not just a notebook experiment).
Production-grade AI engineering practices	Structured JSON outputs, prompt versioning, input validation, and error handling are built into the pipeline (see Section 6).
Ability to explain projects confidently	Every design decision below has a clear "why," so it can be defended in an interview without hand-waving.

3. System Architecture



4. Tech Stack

Component	Tool / Library	Purpose
Language	Python 3.10+	Core implementation
Document Parsing	<code>PyPDFLoader</code> (LangChain)	Extract text from resume PDFs
Text Splitting	<code>RecursiveCharacterTextSplitter</code> (LangChain)	Break resumes into retrievable chunks
Embeddings	<code>sentence-transformers</code> (e.g. <code>all-MiniLM-L6-v2</code>) or OpenAI embeddings	Convert text chunks into vectors
Vector Store	<code>FAISS</code>	Store and retrieve resume chunks relevant to the JD (RAG core)
Orchestration	<code>LangGraph</code>	Model the retrieve → score → summarize steps as an explicit stateful graph
LLM	OpenAI GPT model or open-source model via HuggingFace	Generates match score + skill-gap explanation
Output Format	Structured JSON → rendered as a table	Human-readable ranked output
(Optional) API Layer	<code>FastAPI</code>	Expose the pipeline as an endpoint for a simple frontend

5. Workflow — Step by Step

Step 1: Input

- Recruiter/user provides a Job Description (plain text) and uploads one or more resume PDFs.

Step 2: Parsing & Chunking

- Each resume PDF is loaded and split into overlapping text chunks (e.g. 500 characters, 50-character overlap) so that context isn't lost mid-sentence.

Step 3: Embedding & Retrieval (RAG)

- All resume chunks are embedded and stored in a FAISS index.
- The JD itself is embedded, and used as the query to retrieve the top-k most relevant chunks from each candidate's resume — this ensures the LLM only sees the *most relevant* parts of a (potentially long) resume, not the whole document.

Step 4: LangGraph Node 1 — Retrieve

- Input: JD + candidate's resume chunks.
- Output: a condensed context block containing only the retrieved, JD-relevant resume content.

Step 5: LangGraph Node 2 — Score & Summarize

- Input: JD + retrieved context from Node 1.
- The LLM is prompted (see Section 6) to return a structured JSON with:
 - `match_score` (0-100)
 - `matching_skills` (list)
 - `missing_skills` (list)
 - `summary` (1-2 line verdict)

Step 6: Aggregation & Ranking

- Once every candidate has been processed through the graph, results are sorted by `match_score` descending and displayed as a ranked table.

6. Prompts Used

6.1 System Prompt (used for every LLM call)

You are an AI recruitment assistant helping a hiring team screen resumes against a job description. You must be objective, evidence-based, and concise. Only use information present in the provided resume context — never assume or invent candidate skills or experience that are not explicitly stated. Always respond in valid JSON matching the schema provided in the user prompt. Do not include any text outside the JSON object.

Why this prompt is designed this way:

- "Objective, evidence-based" — reduces hallucination risk, a key production concern.
- "Only use information present in the provided resume context" — explicitly grounds the LLM in the RAG-retrieved content, reinforcing the retrieval step rather than letting the model guess.
- "Always respond in valid JSON... no text outside" — this is what makes the output machine-parseable for ranking/sorting, which is essential for a real application (not just a chat reply).

6.2 User Prompt Template (Node 2 — Score & Summarize)

Job Description:

{job_description}

Candidate Resume Context (retrieved sections most relevant to this JD):

{retrieved_resume_chunks}

Task:

Compare the candidate's resume context against the job description and return a JSON object with exactly this schema:

```
{{
  "match_score": <integer 0-100>,
  "matching_skills": [<list of skills/requirements the candidate clearly
meets>],
  "missing_skills": [<list of skills/requirements from the JD not evidenced
in the resume>],
  "summary": "<one or two sentence verdict for a recruiter>"
}}
```

Rules:

- Base match_score only on overlap between JD requirements and resume evidence.
- If a skill is not mentioned in the resume context, it must appear in missing_skills, even if it seems like a common skill.
- Do not output anything other than the JSON object.

Why this prompt is designed this way:

- Providing the exact JSON schema inline (rather than describing it in prose) significantly reduces malformed output — a common production issue when parsing LLM responses downstream.
- The "Rules" section explicitly closes loopholes (e.g., assuming a common skill is present) that would otherwise let the model be overly generous — improving reliability for a screening use case where false positives are costly.

6.3 Retrieval Query Prompt (Node 1 — used to query FAISS)

```
Query: {job_description}
Top-k: 5
Filter: candidate_id == {current_candidate_id}
```

This isn't an LLM prompt — it's the retrieval query passed to FAISS, restricted per-candidate so resumes don't leak into each other's context.

7. Sample Output

Rank	Candidate	Match %	Missing Skills
1	Priya R.	88%	Docker
2	Arun K.	74%	AWS, Docker
3	Karthik S.	61%	LangChain, AWS, Docker

Each row can be expanded to show the full JSON (matching skills + one-line summary) for recruiter review.

8. Production Engineering Practices Applied

- **Structured output enforcement** — JSON schema in-prompt + a parsing/validation step with retry-on-malformed-output logic.
- **Per-candidate retrieval isolation** — prevents cross-contamination between candidates' resume chunks in the vector store.
- **Prompt versioning** — prompts are stored as separate template files (not hardcoded inline), so they can be updated/A-B tested without touching pipeline code.
- **Error handling** — PDF parsing failures (corrupted files, scanned/image-only resumes) are caught and flagged rather than silently skipped.
- **Logging** — each candidate's retrieval + scoring step is logged with a timestamp for auditability (important in a hiring context).

9. Possible Extensions (good talking points for "future work")

- Add a re-ranking step using a cross-encoder for higher retrieval precision.
 - Support scanned/image resumes via OCR before chunking.
 - Add a second LangGraph branch for interview-question generation based on `missing_skills`.
 - Deploy behind a FastAPI endpoint with a simple React/Streamlit dashboard.
-

10. Key Talking Points for Interview

- **"Why RAG instead of just pasting the whole resume into the prompt?"** → Long resumes exceed useful context and dilute relevance; retrieval keeps the LLM focused on JD-relevant content only, and scales to many long resumes without hitting token limits.
- **"Why LangGraph instead of a single prompt call?"** → Splitting retrieval and scoring into separate graph nodes makes the pipeline debuggable (you can inspect what was retrieved before scoring) and extensible (new nodes, like re-ranking, can be inserted without rewriting the whole flow).
- **"How do you keep the LLM from hallucinating skills?"** → Explicit system-prompt grounding rules + requiring evidence-based JSON output, cross-checked against the retrieved context only.